

ABC by Example and Experiments (Top-Down Approach)

Global and local variables, variable scopes, for-loops, ...

@ <stdio.h>

```
fn foo(val: int): int
{
    while (val < 10) {
        val = val * 3;
    }
    return val;
}

fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

@ <stdio.h>

```
fn foo(val: int): int
{
    local i: int;

    i = 0;
    while (val < 10) {
        val = val * 3;
        i = i + 1;
    }
    printf("i = %d\n", i);
    return val;
}
```

```
fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```


@ <stdio.h>

```
fn foo(val: int): int
{
    local i: int = 0;

    while (val < 10) {
        val = val * 3;
        i = i + 1;
    }
    printf("i = %d\n", i);
    return val;
}
```

```
fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

@ <stdio.h>

```
fn foo(val: int): int
{
    local i: int;

    for (i = 0; val < 10; i = i + 1) {
        val = val * 3;
    }
    printf("i = %d\n", i);
    return val;
}

fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

@ <stdio.h>

```
fn foo(val: int): int
{
    for (local i: int = 0; val < 10; i = i + 1) {
        val = val * 3;
        printf("i = %d\n", i);
    }
    return val;
}
```

```
fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```


@ <stdio.h>

```
fn foo(n: int): int
{
    local result: int = 1;
    for (local i: int = 1; i <= n; i = i + 1) {
        printf("in loop: i = %d\n", i);
        result = result * i;
    }
    return result;
}

fn main()
{
    printf("foo(%d) = %d\n", 0, foo(0));
    printf("foo(%d) = %d\n", 5, foo(5));
}
```

@ <stdio.h>

```
fn foo(n: int): int
{
    local result: int = 1;
    for (local i: int = 1; i <= n; ++i) {
        printf("in loop: i = %d\n", i);
        result *= i;
    }
    return result;
}
```

```
fn main()
{
    printf("foo(%d) = %d\n", 0, foo(0));
    printf("foo(%d) = %d\n", 5, foo(5));
}
```


@ <stdio.h>

```
fn foo(val: int): int
{
    local i: int = 42;
    printf("before loop: i = %d\n", i);
    for (local i: int = 1; val < 10; ++i) {
        printf("in loop: i = %d\n", i);
        val *= 3;
    }
    printf("after loop: i = %d\n", i);
    return val;
}
```

```
fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

```
@ <stdio.h>
```

```
global i: int = 13;
```

```
fn foo(val: int): int
{
    printf("global i = %d\n", i);
    i = val;
    local i: int = 42;
    printf("before loop: i = %d\n", i);
    for (local i = 1; i < val; ++i) {
        printf("in loop: i = %d\n", i);
    }
    printf("after loop: i = %d\n", i);
    return i;
}
```

```
fn main()
{
    printf("foo(%d) = %d\n", 1, foo(1));
    printf("foo(%d) = %d\n", 2, foo(2));
}
```

```
@ <stdio.h>
```

```
global
```

```
    n: int,  
    result: int;
```

```
fn foo()
```

```
{  
    result = n * n;  
}
```

```
fn main()
```

```
{  
    n = 1;  
    foo();  
    printf("result = %d\n", result);  
    n = 2;  
    foo();  
    printf("result = %d\n", result);  
}
```



```
@ <stdio.h>
```

```
global
```

```
    foo_n: int,  
    foo_result: int;
```

```
fn foo()
```

```
{  
    if (foo_n > 1) {  
        foo_result *= foo_n;  
        --foo_n;  
        foo();  
    }  
}
```

```
fn main()
```

```
{  
    foo_n = 0;  
    foo_result = 1;  
    foo();  
    printf("result = %d\n", foo_result);  
    foo_n = 5;  
    foo_result = 1;  
    foo();  
    printf("result = %d\n", foo_result);  
}
```

Operator

Equivalent

a += b

a = a + b

a -= b

a = a - b

a *= b

a = a* b

a /= b

a = a / b

a %= b

a = a % b

++a

a = a + 1

--a

a = a - 1